SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Operating Systems **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO4: *Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

Dr VIMAL V R

QUESTIONS: 5 **Total Marks:25**

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

I _Narmatha J (192521341)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Narmatha J

Annexure A

COMPARATIVE ANALYSIS

S.No	Comparative Analysis Questions	Marks
1	Compare **Contiguous and Linked file allocation methods** in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.	5
2	Differentiate between **Linked and Indexed file allocation** with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.	5
3	Compare **FCFS and SSTF disk scheduling algorithms** based on seek time, fairness, andthepossibilityofstarvation.Discusswhichalgorithmperformsbetterunderheavy disk load.	5
4	Analyzethedifferencesbetween**SCANandC-SCANdiskschedulingalgorithms**in terms of head movement, response time, and uniformity of service. Which algorithm Is more suitable for real-time systems and why?	5
5	Compare **LOOK and C-LOOK disk scheduling algorithms** with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

ANSWERS:

1. Contiguous vs Linked File Allocation

Feature	Contiguous Allocation	Linked Allocation
Storage	File blocks are stored continuously	Blocks can be stored anywhere
Access Speed	Very fast, especially for sequential and direct access	Slower because links must be followed
Memory Utilization	May waste space due to external fragmentation	Better utilization of free space
Fragmentation	Suffers from external fragmentation	No external fragmentation
Sequential Access	Very efficient	Efficient but slower than contiguous

Suitable method for sequential access:

Contiguous allocation is more suitable because all file blocks are stored next to each other. The disk head can read the blocks continuously with less movement, resulting in faster access and better performance.

2. Linked vs Indexed File Allocation

Feature	Linked Allocation	Indexed Allocation
Random Access	Poor because blocks must be followed one by one	Good because the index directly points to file blocks
Storage Overhead	Requires pointers in each block	Requires a separate index block

Reliability	If a pointer is damaged, part of the file may become inaccessible	More reliable because block addresses are maintained in the index
File Access	Mainly suitable for sequential access	Suitable for both sequential and random access
Large Files	Pointer overhead increases	Can efficiently manage large files using multiple index blocks

More efficient for large files:

Indexed allocation is generally more efficient for large files because it provides direct access to blocks and avoids following a long chain of pointers. It is especially useful when random access to large files is required.

3. FCFS vs SSTF Disk Scheduling

Feature	FCFS	SSTF
Meaning	Requests are served in arrival order	Request closest to current head position is served first
Seek Time	Usually high	Usually lower
Fairness	Very fair because requests follow arrival order	Less fair
Starvation	No starvation	Starvation is possible
Performance	Simple but may be slower	Usually faster

Performance under heavy disk load:

SSTF generally performs better because it selects the request closest to the current head position, reducing average seek time. However, under heavy

load, requests located far from the current head may wait for a long time and can suffer from starvation.

Therefore, FCFS is better for fairness, while SSTF is better for average seek performance.

4. SCAN vs C-SCAN Disk Scheduling

Feature	SCAN	C-SCAN
Head Movement	Moves in both directions like an elevator	Moves in one direction only
Response Time	Response time can vary depending on request position	More uniform response time
Service	Requests are served in both directions	Requests are served only while moving in one direction
Efficiency	Good efficiency	Slightly more head movement
Fairness	Fair	More uniform/fair for different positions

Suitable for real-time systems:

C-SCAN can be more suitable when predictable and uniform response time is important. Since the disk head services requests in one direction and then returns to the beginning, requests receive more consistent service.

However, the choice depends on the real-time requirements. If minimizing total head movement is more important, SCAN may be preferred.

5. LOOK vs C-LOOK Disk Scheduling

Feature	LOOK	C-LOOK
Direction	Moves in both directions	Moves in one direction
Head Movement	Stops at the last request before reversing	Jumps back after reaching the last request
Efficiency	Efficient because it does not go to disk ends unnecessarily	Very efficient for uniform service
Implementation	Relatively simple	Slightly more complex
Performance	Good for general workloads	Better when uniform response time is required

Better performance:

C-LOOK can provide better performance when requests are spread across the disk and uniform waiting time is important. It avoids unnecessary movement to the physical ends of the disk.

LOOK is preferable when the main goal is to reduce total head movement and the workload is not highly demanding. Both are improvements over SCAN and C-SCAN because they stop at the last pending request instead of travelling all the way to the disk boundary.